

1. 배열과 함수.

- 배열은 함수에 넘기면 복사 X
- 시작 주소값만 전달.

⇒ 함수 안에서 값 바꾸면 원본 바꾸!

(array[] == *array : 컴파일시 동일하게 작동)

2. 배열 크기 문제

- 함수는 주소만 받아서 배열이 몇칸인지 모름
- 크기 넘겨주지 않으면 빛의 초과 접근시 컴파일러 실행 오류.
- 해결: 매개변수로 크기(n)도 같이 전달.

3. sizeof 연산자

- sizeof(변수/배열) → 바이트 크기 반환

ex) `int list[5];`
`sizeof(list);` // 20
`sizeof(list[0]);` // 4
개수 = `sizeof(list) / sizeof(list[0]);` // 5
△ `int* array` 쓴 경우 그냥 8 나옴

4. 포인터 크기

- 포인터 = 주소값 하나 담은 변수 → 항상 8 바이트
- `int*`, `char*`, `double*` 무엇을 가리키는 크기는 동일.

5. 2차원 배열과 함수

- `a[i][j]` 주소 = 시작주소 + (`i` x 열개수 + `j`) x 4 byte

⇒ 따라서 열개수는 함수선언에 반드시 명시!

ex) `void myPrint(int a[][3], int row, int col) { ... }`

6. 포인터와 문자열

ex) `char s1[] = "Hello";` // 배열 → 수정 가능
`char* s2 = "Hello";` // 포인터 → 수정 불가

- `'\0'`: 문자열 끝 표시 (널 종료 문자)

- `%s`: `'\0'` 나올 때까지 자동 출력 / `%c`: 한 글자씩 (for 문 필요)

7. 문자열 입력

ex) `scanf("%s", s1);` // 공백에서 멈춤

`scanf("%[^\n]", s1);` // 줄바꿈 전까지 공백 포함 전부 입력

8. 이중 포인터

- 포인터가 "가리키는 곳 자체"를 함수 안에서 바꾸려면 ** 필요

0. 17 → 1~1

- 포인터가 "가리키는 곳 자체"를 함수 안에서 바꾸려면 ** 필요

ex) `void GetStr(char** str)`

`{ *str = "I am happy";`

`} GetStr(&s); // s의 주소를 넘김.`

- 이유: 포인터도 결국 변수라서 그냥 넘기면 복사본만 바꾸고.

7. 함수 포인터

- 선언 : `int (*fp)(int, int);` // "int를 반환하고, int 2개를 받는 함수"를 가리키는 포인터

- `int` → 함수가 반환하는 타입

- `(*fp)` → fp가 포인터라는 뜻

- `(int, int)` → 이 함수가 받는 인자들의 타입

ex) `int add(int a, int b) {return a+b;}`

`int main(void)`

`{ int (*fp)(int, int);`

`fp = add; // add 함수의 주소를 fp에 저장`

`int result = fp(3,4); // add(3,4)랑 똑같이 동작 result = 7`

`}`

8. 구조체 포인터

- `struct Point p = {10, 20};`

`struct Point** pp = &p; // p의 주소를 pp에 저장`

- 접근 방법:

① $(*PP).X$

② $PP \rightarrow X$

ex) void move (struct point* PP, int dx, int dy)

{
PP->X += dx;
PP->Y += dy;
}

int main(void)

{ struct point p = {10, 20};

move(&p, 5, 5); // p의 주소를 넘김

printf("%d %d", p.x, p.y); // 15 25 - 원본이 간접으로 바뀌기

{